Program 1

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>
#define SIZE 100000
void merge(int a[], int l, int m, int r) {
  int n1 = m - l + 1, n2 = r - m, i = 0, j = 0, k = l;
  int *L = malloc(n1 * sizeof(int)), *R = malloc(n2 * sizeof(int));
  for (int x = 0; x < n1; x++) L[x] = a[l + x];
  for (int y = 0; y < n2; y++) R[y] = a[m + 1 + y];
  while (i < n1 && j < n2) a[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
  while (i < n1) a[k++] = L[i++];
  while (j < n2) a[k++] = R[j++];
  free(L); free(R);}
void serialMergeSort(int a[], int l, int r) {
  if (l < r) {
    int m = l + (r - l) / 2;
    serialMergeSort(a, l, m);
    serialMergeSort(a, m + 1, r);
    merge(a, l, m, r);
  }}
void parallelMergeSort(int a[], int l, int r, int depth) {
  if (l < r) {
    int m = l + (r - l) / 2;
    if (depth <= 4) {
      #pragma omp parallel sections {
        #pragma omp section
        parallelMergeSort(a, l, m, depth + 1);
        #pragma omp section
```

```
            parallelMergeSort(a, m + 1, r, depth + 1);
        } } else {
        serialMergeSort(a, l, m);
        serialMergeSort(a, m + 1, r);}
      merge(a, l, m, r);
  }}
int main() {
  int *s = malloc(SIZE * sizeof(int)), *p = malloc(SIZE *
sizeof(int));
  for (int i = 0; i < SIZE; i++) s[i] = p[i] = rand() % 100000;
  clock_t t = clock();
  serialMergeSort(s, 0, SIZE - 1);
  double time_serial = (double)(clock() - t) /
CLOCKS_PER_SEC;
  t = clock();
  parallelMergeSort(p, 0, SIZE - 1, 0);
  double time_parallel = (double)(clock() - t) /
CLOCKS_PER_SEC;
  printf("Serial Merge Sort Time: %.6f seconds\n", time_serial);
  printf("Parallel Merge Sort Time: %.6f seconds\n",
time_parallel);
  free(s); free(p);
  return 0;}
```

Program 2

```
#include<stdio.h>
#include<omp.h>
int main()
{
int n = 16,thread,i;
printf("\n Enter the number of tasks");
scanf("%d",&n);
printf("\n n Enter the number of threads");
scanf("%d",&thread);
omp_set_num_threads(thread);
printf("\n ----------------------------------------------------------------
------\n");
#pragma omp parallel for schedule(static, 2)
for (i = 0; i< n; i++)
{
printf("Thread %d executes iteration %d\n",
omp_get_thread_num(), i);
}return 0;}
```

Program 3

```
#include <stdio.h>
#include <omp.h>
// Serial Fibonacci calculation function
int ser_fib(long int n){
   if (n < 2)
      return n;
   return ser_fib(n - 1) + ser_fib(n - 2);}
// Parallel Fibonacci calculation function
int fib(long int n){
   if (n < 2)
      return n;
   int x, y;
   #pragma omp task shared(x)
   x = fib(n - 1);
   #pragma omp task shared(y)
   y = fib(n - 2);
   #pragma omp taskwait
   return x + y;
}
int main(){
   long int n = 10;
   int result;
   double start, end;
   double parallel_time, serial_time;
   printf("Enter the value of n: ");
   scanf("%ld", &n);
   // Parallel Fibonacci
   start = omp_get_wtime();
```

```
   #pragma omp parallel {
      #pragma omp single  {
         result = fib(n);  } }
   end = omp_get_wtime();
   parallel_time = end - start;
   printf("\nParallel Fibonacci(%ld) = %d", n, result);
   printf("\nTime used in parallel mode: %f seconds\n",
parallel_time);
   // Serial Fibonacci
   start = omp_get_wtime();
   result = ser_fib(n);
   end = omp_get_wtime();
   serial_time = end - start;
   printf("\nSerial Fibonacci(%ld) = %d", n, result);
   printf("\nTime used in serial mode: %f seconds\n",
serial_time);
   return 0; }
```

Program 4

```
#include <stdio.h>
#include <omp.h>
int is_prime(int n)  {
   if (n < 2)
      return 0;
   for (int j = 2; j * j <= n; j++)  {
      if (n % j == 0)
         return 0; }
   return 1;}
int main() {
   long n = 10000000, i;
   long serial_count = 0, parallel_count = 0;
   double start, end;
   double serial_time, parallel_time;
   printf("Enter the range of number: ");
   scanf("%ld", &n);
   printf("\nThe range of numbers is 1 to %ld\n", n);
   printf("\n-------------------------------------------------\n");
   // Serial Execution
   start = omp_get_wtime();
   for (i = 0; i <= n; i++)   {
      if (is_prime(i))
         serial_count++;    }
   end = omp_get_wtime();
   serial_time = end - start;
   printf("Serial prime count = %ld\n", serial_count);
   printf("Time to compute prime numbers serially: %f
seconds\n", serial_time);
```

```
 // Parallel Execution
   start = omp_get_wtime();
   #pragma omp parallel for reduction(+:parallel_count)
   for (i = 0; i <= n; i++)   {
      if (is_prime(i))
         parallel_count++;   }
   end = omp_get_wtime();
   parallel_time = end - start;
   printf("Parallel prime count = %ld\n", parallel_count);
   printf("Time to compute prime numbers in parallel: %f
seconds\n", parallel_time);
   return 0; }
```